Mikhail Razakov

NET Backend / Platform Engineer · State & Reliability

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

I build .NET backend systems where payments, access, and state must survive retries and restarts: explicit state machines, provider re-verification, durable inbox/outbox, idempotency, reconciliation, migrations, and verifiable rollback

Payment correctness

A webhook is not treated as ground truth: provider payment state is re-fetched and checked for id/status/environment/metadata/amount/currency

Recovery ownership

Durable inbox/outbox, idempotency, reconciliation, restart recovery, leases, and manual review for conflicting evidence

Release safety

Versioned releases, coordinated backup, atomic switching, health gates, and rollback to the previous release

EXPERIENCE

2026 — present

CompilationLabLMS — payment/backend architecture

- Made PaymentOrder the owner of allowed state transitions, retry scheduling, and status-check leases; the webhook flow re-fetches payment state from the provider before emitting an internal event
- Validate payment id/status/environment/metadata/amount/currency and keep malformed or unverifiable provider events separate from confirmed state

2026 — present

VpnMediator — state/recovery backend

- Designed a durable payment inbox/outbox, idempotent reconciliation, and a manual-review path for conflicting provider evidence; restart recovery is covered by dedicated tests
- Built a versioned release path with coordinated backup, atomic switching, readiness/liveness gates, and restoration of the previous release after failed startup

2024 — present

UniversalToolchain — .NET platform architecture

- Apply the same reliability discipline to platform code through single-owner planning, exact package binding, and regression contracts; the current exact manifest is 1,306/1,306 passing

PROJECTS

CompilationLabLMS

The architecture separates user workflows from financial invariants and supports safe migration and rollback.

VpnMediator

A production service with controlled state transitions, backup/restore, health gates, and safe rollback; the public case study excludes secrets and user data.

UniversalToolchain

The modular framework separates language composition from runtime execution and backs its contracts with executable verification/tests.

SKILLS

NET Backend

C#/.NET, ASP.NET Core, REST/OpenAPI, webhooks, background services, provider integrations

State & Data

PostgreSQL, SQLite, transactions, explicit state machines, idempotency keys, inbox/outbox, leases

Reliability

provider re-verification, reconciliation, retry ownership, restart recovery, audit/manual review, backup/restore

Operations / Platform

Linux, Docker Compose, nginx, systemd, health gates, versioned releases, atomic switching, rollback

EDUCATION

Software Engineering student at HSE University

RECOGNITION

First-degree diploma and the Grand Prize “Perfection as Hope” for UniversalToolchain

Moscow / remote